



中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

一、实验题目

房价预测任务的感知机算法

data.csv 数据集包含五个属性,共 10000 条数据,其中 longitude 和 latitude 表示房子经纬度, housing_age 表示房子年龄, homeowner_income 表示房主的收入 (单位: 十万元), house_price 表示房子的价格。

请根据数据集 data.csv 中四个属性 longitude、latitude、housing_age、homeowner_income, 利用感知机算法预测房价 house_price, 并画出数据可视化图、loss 曲线图。

二、实验内容

1. 算法原理

1. 原理说明:

多层感知机 MLP 是一种经典的前馈神经网络,是深度学习的基础。它由多个神经元层组成,每一层都与下一层全连接。MLP 可以学习复杂的非线性关系,因此适用于各种回归和分类任务。

2. 基本结构:

输入层: 接收输入特征。本题中, 输入层接收 longitude、latitude、housing_age 和 homeowner_income 这 4 个特征。

隐藏层: 包含多个神经元,每个神经元都对输入进行加权求和,并通过激活函数进行非线性转换。MLP 可以包含多个隐藏层,增加模型的复杂度。

输出层: 产生最终的预测结果。本题中, 输出层只有一个神经元, 预测房价 house_price。

3. 神经元

每个神经元接收来自上一层的所有输入,并为每个输入分配一个权重。

神经元将加权后的输入求和,并加上一个偏置项。

神经元将求和结果通过激活函数进行非线性转换。常用的激活函数包括 Sigmoid、ReLU 等。

4. 前向传播

输入数据从输入层开始,逐层传递到隐藏层,最后到达输出层。

每一层都对输入进行加权求和和激活函数转换。
输出层产生最终的预测结果。

5. 反向传播

计算预测结果与真实值之间的损失（Loss）。
本题中可以使用均方误差（MSE）作为损失函数。
计算损失函数对每个权重的梯度。
使用梯度下降等优化算法更新权重，以减小损失。

6. 训练过程

重复前向传播和反向传播，直到模型收敛或达到最大迭代次数。
通过训练，MLP 可以学习输入特征与输出目标之间的复杂关系。

2. MLP 在本题中的应用

1. 数据准备

特征选择: 选择 longitude、latitude、housing_age 和 homeowner_income 作为输入特征。
数据预处理: 对数据进行标准化或归一化，以提高模型训练的效率和准确性。

2. 模型构建

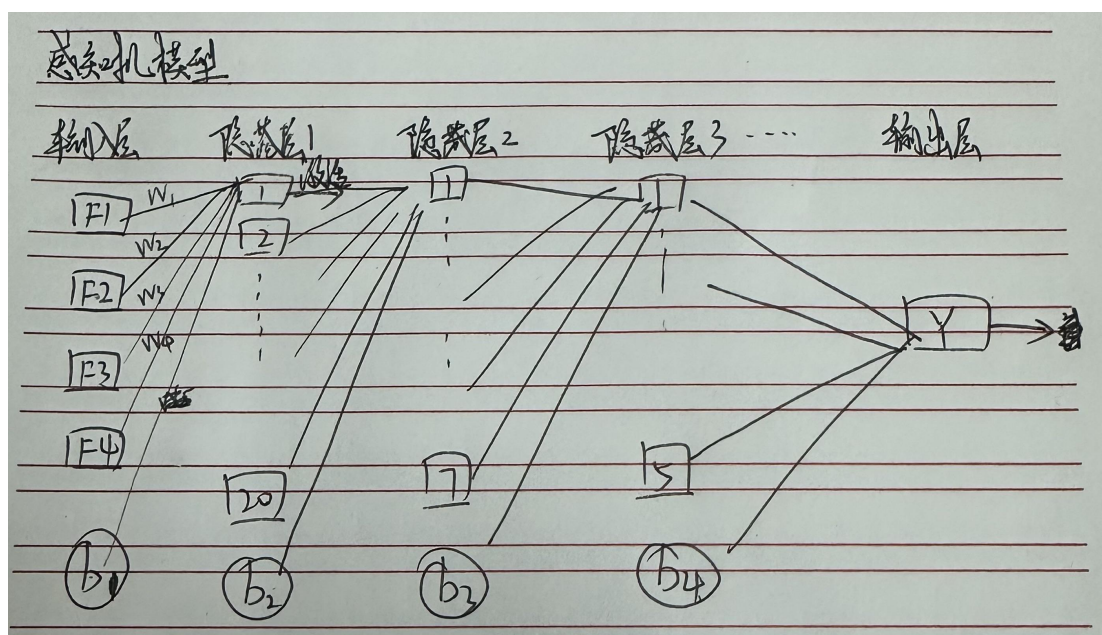
确定网络结构: 确定 MLP 的层数和每层的神经元数量。
选择激活函数: 选择合适的激活函数。隐藏层通常使用 ReLU，输出层使用 Sigmoid。
初始化权重: 使用合适的权重初始化方法，防止梯度消失，在这里使用 He 初始化。

3. 模型训练

定义损失函数: 使用均方误差（MSE）作为损失函数。
选择优化算法: 使用梯度下降等优化算法更新权重。
设置超参数: 设置学习率、迭代次数等超参数。
训练模型: 使用训练集训练 MLP 模型。
绘制损失曲线: 绘制损失曲线，观察模型的训练过程。

4. 例子

隐藏层有四个输入特征
中间层有三个隐藏层，每一层的神经元个数不同
输出层为房价的预测值
每一个神经元有权重系数和偏置数
神经元经过线性求和，再经过激活函数向下一层传递



3. 伪代码

导入库 (numpy, pandas, sklearn)

定义激活函数 $\text{sigmoid}(Z)$: 返回 $\text{sigmoid}(Z)$, Z

定义激活函数 $\text{relu}(Z)$: 返回 $\max(0, Z)$, Z

定义 $\text{Init_Parameters}(\text{layers_dims})$:

初始化 W, b (He 初始化)

返回 parameters

定义 $\text{update_parameters}(\text{parameters}, \text{grads}, \text{learning_rate})$:

更新 W, b

返回 parameters

定义 $\text{linear_forward}(A, W, b)$: 返回 $Z, (A, W, b)$

定义 $\text{activation_forward}(A_{\text{prev}}, W, b, \text{activation})$:

$\text{linear_forward} + \text{activation}$ (sigmoid 或 relu)

返回 A, cache

定义 $\text{Model_Forward}(X, \text{parameters})$:

循环 layers: $\text{linear_activation_forward}(\text{relu})$

$\text{linear_activation_forward}(\text{sigmoid})$

返回 AL, caches

定义 backward 函数 (sigmoid, relu, linear, linear_activation): 计算梯度



定义 `Model_Backward(AL, Y, caches):`

计算所有梯度

返回 `grads`

定义 `MLP_Model(X, Y, layers_dims, learning_rate, Iterators_nums):`

初始化 `parameters`

循环 `Iterators_nums:`

前向传播, 反向传播, 更新 `parameters`

返回 `parameters`

定义 `read_data(filename, samples_num):` 读取数据, 返回 `X, Y`

定义 `dif(Y, price):` 打印原值, 预测值, 差异

主程序:

读取数据, 预处理 (归一化)

训练模型

预测, 反归一化

可视化

4. 关键代码展示 (带注释)

1. 数据读取

```
# 数据读取
def read_data(filename, samples_num):
    data = pd.read_csv(filename, sep=",", header=0)
    title = list(data.columns)
    data = data[title]
    data = data[0:samples_num]
    X = data.iloc[:, :-1].values
    Y = data.iloc[:, -1].values
    return X, Y
```

2. 数据预处理

```
#数据预处理、归一化
features_num = X.shape[1]
scaler = StandardScaler()
X = scaler.fit_transform(X).T
scaler_Y = MinMaxScaler(feature_range=(0, 1))
Y = scaler_Y.fit_transform(Y_real.reshape(-1, 1)).T
```



3. 主函数

```
#模型训练

layers_dims = [features_num, 20, 7, 5, 1] #感知机层数和每层神经元个数

parameters, AL_list= MLP_Model(X, Y, layers_dims, learning_rate=0.5, Iterators_nums=1000)

# 反归一化 AL_list 并计算损失
loss_list = []
for AL in AL_list:
    price_pre = scaler_Y.inverse_transform(AL.T).T #反归一化
    price_real= scaler_Y.inverse_transform(Y)
    loss = np.mean((price_pre-price_real)**2) #均方误差损失
    loss_list.append(loss)

# 绘制损失曲线
plt.plot(loss_list)
plt.ylabel('Loss')
plt.xlabel('Iterations')
plt.title('Loss curve')
plt.show()
```

4. 激活函数

```
# 激活函数
def sigmoid(Z):
    A = 1 / (1 + np.exp(-Z))
    return A, Z # 返回用于反向传播的缓存值 Z

def relu(Z):
    A = np.maximum(0, Z) # ReLU 激活函数
    return A, Z # 返回用于反向传播的缓存值 Z
```

5. 线性函数

```
def linear_forward(A, W, b):
    Z = np.dot(W, A) + b
    assert Z.shape == (W.shape[0], A.shape[1])
    cache = (A, W, b) #缓存值包含前一层的激活值 A、权重 W 和偏置 b
    return Z, cache
```

6. 损失函数

```
# 计算均方误差损失函数的值
def compute_cost(AL, Y):
    cost = np.sum((AL - Y) ** 2) / 2 # 均方误差公式
    cost = np.squeeze(cost) # 确保 cost 是一个标量
    assert cost.shape == ()
    return cost
```



7. 线性函数、激活函数的梯度计算

```
#反向传播
def sigmoid_backward(dA, cache):
    Z = cache
    s = 1 / (1 + np.exp(-Z))
    dZ = dA * s * (1 - s)
    return dZ

def relu_backward(dA, cache):
    Z = cache
    dZ = np.array(dA, copy=True)
    dZ[Z <= 0] = 0
    return dZ

def linear_backward(dZ, cache):
    A_prev, W, b = cache
    m = A_prev.shape[1]
    dW = np.dot(dZ, A_prev.T) / m
    db = np.sum(dZ, axis=1, keepdims=True) / m
    dA_prev = np.dot(W.T, dZ)
    return dA_prev, dW, db
```

8. 初始化参数与更新参数

```
# 初始化参数
def Init_Parameters(layers_dims):
    parameters = {}
    for i in range(1, len(layers_dims)):
        parameters["W" + str(i)] = np.random.randn(layers_dims[i], layers_dims[i - 1]) * np.sqrt(2 / layers_dims[i - 1]) #初始化 防止梯度
        parameters["b" + str(i)] = np.zeros((layers_dims[i], 1)) # 偏置初始化为零
    return parameters

# 参数更新
def update_parameters(parameters, grads, learning_rate):
    L = len(parameters) // 2
    for l in range(L):
        parameters["W" + str(l + 1)] -= learning_rate * grads["dW" + str(l + 1)]
        parameters["b" + str(l + 1)] -= learning_rate * grads["db" + str(l + 1)]
    return parameters
```

9. MLP 模型以及结果可视化

```
#MLP 模型训练
def MLP_Model(X, Y, layers_dims, learning_rate, Iterators_nums):
    parameters = Init_Parameters(layers_dims) #初始化参数
    AL_list = [] #预测值列表
    for i in range(Iterators_nums):
        AL, caches = Model_Forward(X, parameters) #向前传播
        AL_list.append(AL) #记录预测值
        grads = Model_Backward(AL, Y, caches) #向后传播
        parameters = update_parameters(parameters, grads, learning_rate) #更新参数
    return parameters, AL_list #返回训练好的参数和预测值
```

三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

参数设置：

- 1.对 MLP_data.csv 的 10000 条数据进行 MLP 训练
- 2.设置三层隐藏层，神经元数量各自为 20，7，5
- 3.学习率为 0.5
- 4.迭代次数为 1000

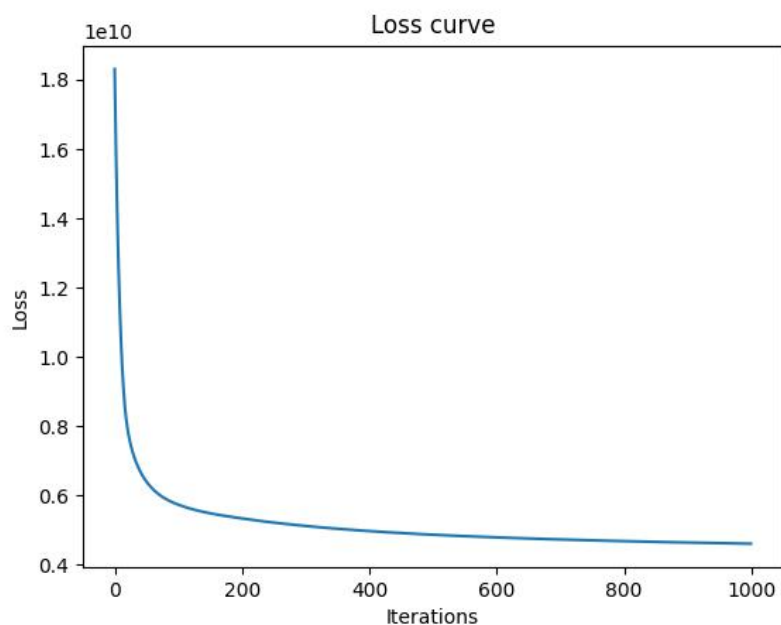
输出结果：

1. 损失值变化

采用的是均方误差损失计算

如果预测价格和原价格相差 10000，则均方误差计算将得到 $10000*10000/2=5*10^7$

在 200 次迭代后趋于平稳





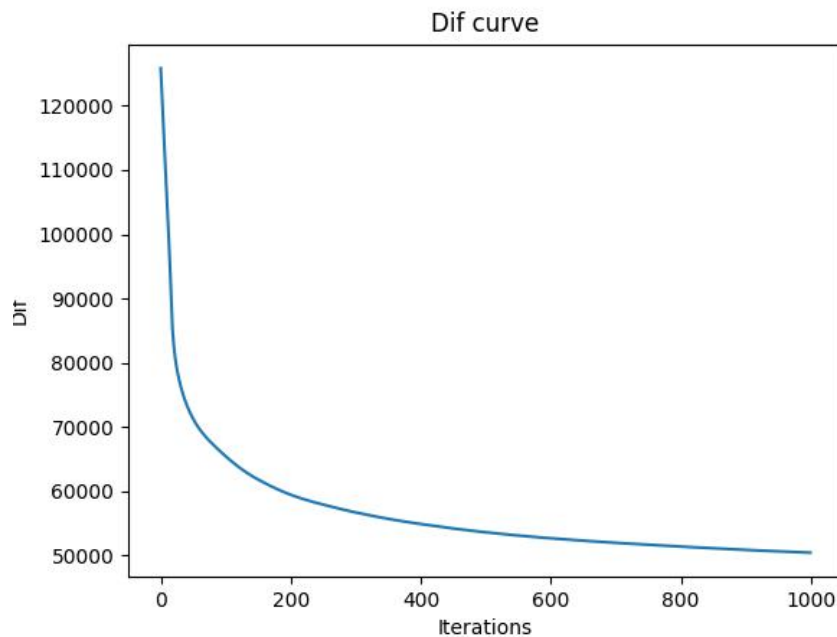
2. 预测偏差变化

这里也同时给出平均预测误差的结果可视化

表示预测的平均价格误差，更加直观和便于理解

第一次训练得到的结果平均误差在 50000 元左右

即表示模型的每一次预测的平均误差范围在原价格的上下 50000 元
在 200 次迭代后趋于平稳



3. 预测的准确率

假设预测的误差小于 10000 则认定为预测准确，则

预测的准确率：14.47%

4. 预测结果：

原值: 452600	预测值: 432737	差异: 19862
原值: 358500	预测值: 400637	差异: -42137
原值: 352100	预测值: 380111	差异: -28011
原值: 341300	预测值: 290996	差异: 50303
原值: 342200	预测值: 212735	差异: 129464
原值: 269700	预测值: 220501	差异: 49198
原值: 299200	预测值: 207333	差异: 91866
原值: 241400	预测值: 183325	差异: 58074
原值: 226700	预测值: 119915	差异: 106784
原值: 261100	预测值: 208323	差异: 52776



2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

1. 预测偏差变化

平均预测误差约为 50,000 元，这表明模型的初始预测并不精准。

随着训练迭代，预测误差逐渐减少并趋于平稳，表明模型在后期已逐渐优化，拟合能力增强。这种现象通常说明模型已经有效捕捉到了数据中的规律和特征，因此它能够更好地预测目标值。

2. 预测的准确率

准确率为 14.47%，假设误差小于 10000 元为准确预测。在这个情况下，预测准确率较低，表明虽然模型有所优化，但它仍存在较大的误差范围。

这可能是由于数据中存在噪声或者输入特征和目标输出之间的非线性关系，导致即使经过较长时间的训练，模型在一些情况下仍未能很好地捕捉到实际趋势。

3. 预测结果的具体分析

通过对比原值和预测值的差异分析：

正差异和负差异：在一些预测结果中，模型的预测值偏高（如原值 452600 预测值 432737，差异 19862），而在其他情况下，预测值偏低（如原值 342200 预测值 212735，差异 129464）。这可能是由于数据的某些特征未能很好地被模型捕捉，导致预测的偏差在某些情况下显著。较大的差异：如原值 342200 与预测值 212735 的差异为 129464，这个误差较大，可能意味着模型在某些特定数据上的拟合效果不佳，可能是由于数据中某些特征没有被有效学习到。

4. 总结

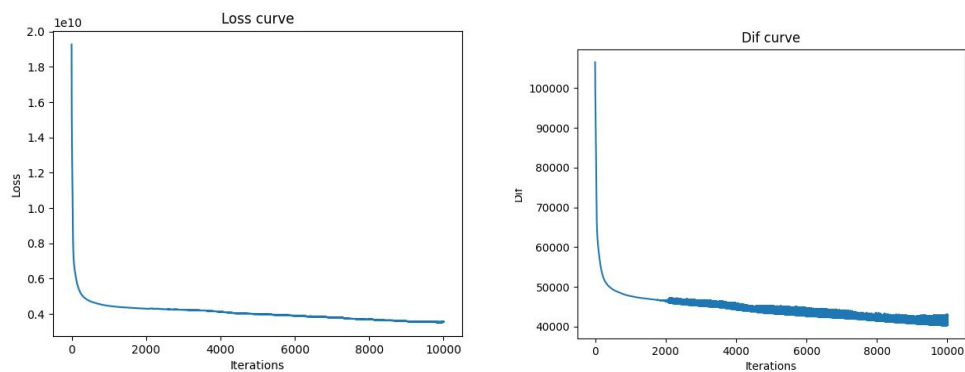
优点：模型在大多数情况下能够较好地拟合数据，并在 200 次迭代后稳定下来，表明训练过程较为有效。损失值和预测偏差在训练过程中逐渐减小，表明模型正在向较优解收敛。

缺点：低准确率：14.47%的准确率较低，表明模型的预测性能仍有待提高。部分预测结果的差异较大，可能是因为数据中存在复杂的非线性关系，或者模型没有捕捉到数据的所有关键特征。

优化一、优化参数

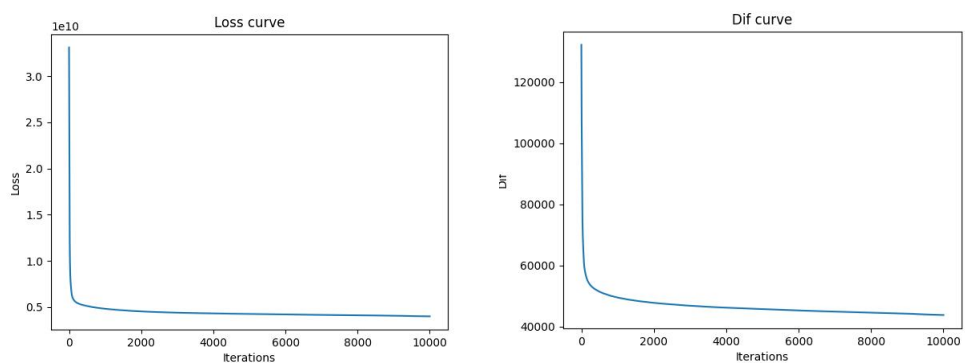
在迭代次数 10000 隐藏层 20, 7, 5 的情况下
不同学习率的结果:

1. 学习率为 1



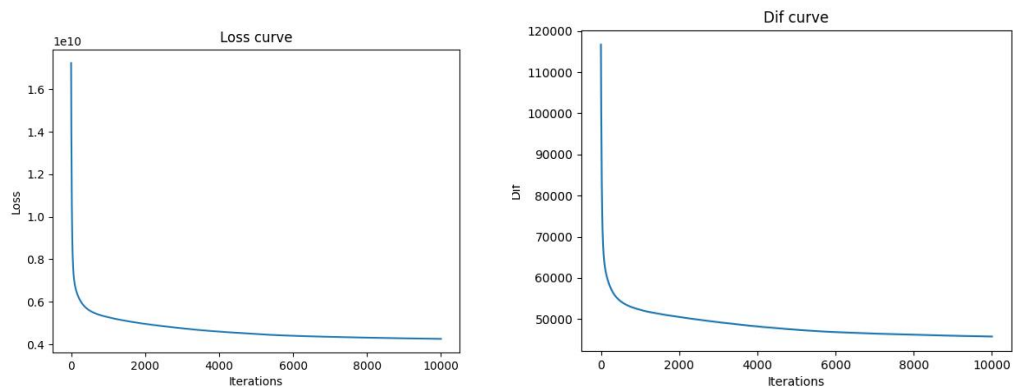
房价偏差从 110000 到 40000
预测的准确率: 20.44%

2. 学习率 0.5



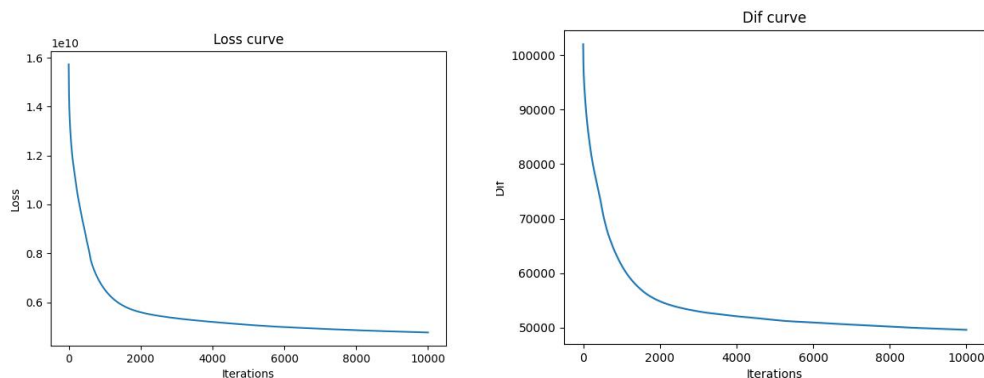
预测的准确率: 19.25%
房价偏差从 110000 到 42000

3. 学习率 0.25



预测的准确率: 16.82%
房价偏差从 115000 到 45000

4. 学习率 0.1



预测的准确率：14.68%

房价偏差从 100000 到 50000

5. 总结

由上面几个例子可知学习率为 1 时效果较好。

学习率大于 1 时，由于更新过于剧烈，模型虽然快速开始，但最终效果不稳定。可能会错过更精细的调整，导致模型预测效果不佳。

学习率为 1 时，模型训练稳定，准确率相对较高。它表现出了较好的平衡，说明此学习率在您的任务中可能是最佳选择。

学习率小于 1 时，模型训练虽然稳定，但速度较慢，且可能未能达到较好的拟合效果，导致准确率下降。尤其在时间有限的情况下，可能需要更多的训练迭代才能看到效果。

优化二、数据处理

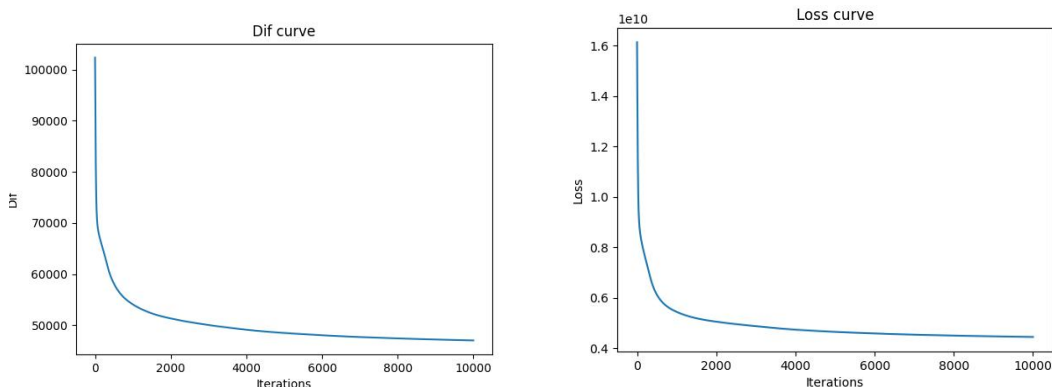
原先的 loss 使用的是归一化后的数据进行计算，可能不够准确。

将计算后得到的归一化的结果先反归一化回原值（即房价），再进行后续处理。此部分的修改已经在代码中体现。

优化三、优化多层感知机的神经元层数和个数

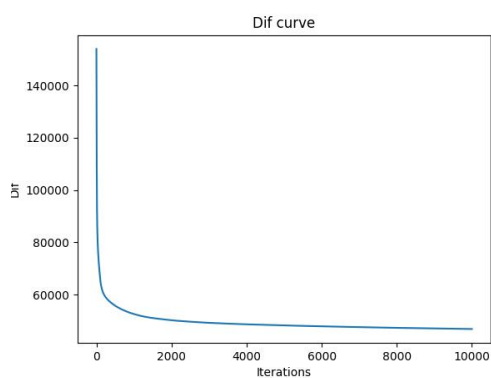
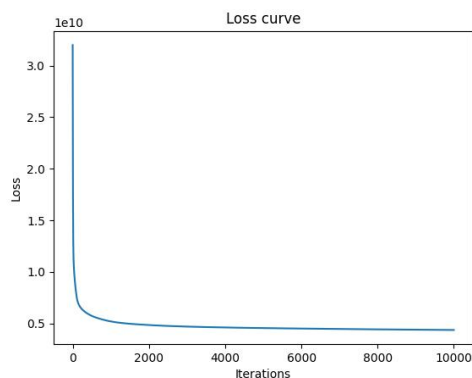
在学习率为 0.1，迭代次数为 10000 下

1. 四层 20, 12, 7, 5



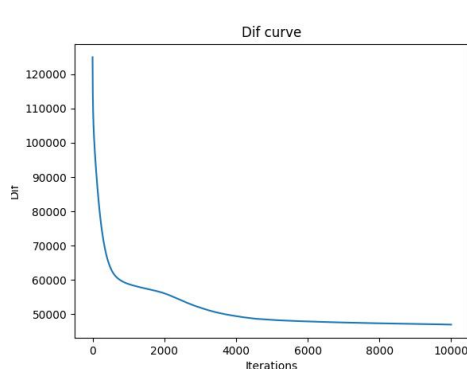
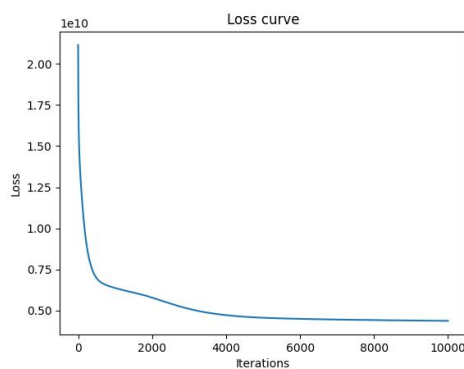
预测的准确率：16.5400%

2. 四层 35, 25, 15, 5,



预测的准确率: 15.4800%

3. 五层 20,15,11,8,3



预测的准确率: 15.3400%

4. 总结

感知机层数和神经元数目影响运行时间和评估结果

综合考虑使用三层感知机 20,7,5。

三层网络表现出了较好的平衡, 既能够捕捉到必要的特征, 又能避免不必要的复杂性。较深的网络可能并未显著提升准确率, 反而增加了计算和训练时间。

四、 思考题

1. 感知机原理与任务适配

感知机算法本质上是用于线性可分问题的分类模型。对于房价预测这样的回归任务，感知机的结构需要调整。可以通过在输出层使用线性激活函数，而非分类激活函数，来使感知机适应回归问题。但考虑到房价预测的复杂性，可能需要引入更多的层数或采用更复杂的回归模型来提高预测精度。

2. 数据预处理与特征工程

由于数据中包含经纬度、房屋年龄和收入等特征，特征的尺度差异较大，因此对数据进行标准化或归一化非常重要，这可以加速模型的收敛。同时，可以通过特征交互或对数变换来优化模型的表达能力，尤其是在房价的分布较为偏态时，进行对数变换可以帮助改善模型拟合。

3. 模型评估与优化

在训练过程中，观察损失曲线有助于判断模型是否收敛，并避免过拟合或欠拟合。同时，可以引入如准确率等评估指标，全面评估模型性能。此外，数据可视化有助于直观了解模型的预测效果，并根据实际需求进一步优化模型结构和训练过程。

五、 参考资料

1. <https://blog.csdn.net/JasonH2021/article/details/131021534>
2. https://tf.wiki/zh_hans/basic/models.html
3. <https://blog.csdn.net/qs17809259715/article/details/100623719>
4. https://blog.csdn.net/2401_84166327/article/details/138493047